Chapter 7
Network Topology Inference

7.1 Introduction

Network graphs are constructed in all sorts of ways and to varying levels of completeness. In some settings, there is little if any uncertainty in assessing whether or not an edge exists between two vertices and we can exhaustively assess incidence between vertex pairs. For example, in examining one's own network of Facebook friends, the presence or absence of an edge can be assessed through direct inspection. In other settings, however, constructing a network graph is not so straightforward. We may have information only on the status of some of the potential edges in the network, but not all. Alternatively, we may not even have the ability to directly assess whether or not an edge is present. Rather, it may be that we can only measure vertex or edge attributes that are to some extent predictive of edge status. In such cases, it can be natural to consider the task of constructing a network graph representation from the available data as one of statistical inference.

To be more precise, suppose that, broadly speaking, we have a set of measurements from a system of interest, such as vertex attributes $\mathbf{x} = (x_1, \ldots, x_{N_v})^T$ or binary indicators $\mathbf{y} = [y_{ij}]$ of certain edges but not others, or both $\mathbf{x}$ and $\mathbf{y}$, and we have a collection $\mathscr{G}$ of potential network graphs G. We might then take as our goal to select an appropriate member of $\mathscr{G}$ that best captures the underlying state of the system, based upon the information in the data as well as any other prior information, using techniques of statistical modeling and inference. That is, we might pose the problem as one of *network topology inference.*

In this chapter, we will focus on three particular classes of problems in network topology inference. Each class of problems is somewhat 'canonical' in nature, being easily posed in more than just a single specific network context. More specifically, in Sect. 7.2 we consider the problem of inferring whether or not a pair of vertices does or does not have an edge between them (i.e., inference of 'edge' or 'non-edge' status), using measurements that include a subset of vertex pairs whose edge/non-edge status is already observed. This problem is commonly referred to as link prediction. Next, in Sect. 7.3, we discuss the inference of association networks.

E.D. Kolaczyk and G. Csárdi, *Statistical Analysis of Network Data with R*, Use R! 65,
DOI 10.1007/978-1-4939-0983-4_7,

Here the relation defining edges is taken by definition to be a nontrivial level of association (e.g., correlation) between certain characteristics of the vertices, but is itself unobserved and must be inferred from measurements reflecting these characteristics. Finally, we examine problems of tomographic network inference briefly in Sect. 7.4. These problems are distinguished by the fact that measurements are available only at vertices that are somehow at the 'perimeter' of the network, and it is necessary to infer the presence or absence of both edges and vertices in the 'interior.'

The ordering of these three classes of problems has a general progression. The first assumes knowledge of all of the vertices and the status of some of the edges/non-edges of the network graph, and seeks to infer the rest of the edges/non-edges. The second starts with no knowledge of edge status anywhere in the network graph, but assumes relevant measurements at all of the vertices and seeks to infer edge status throughout the network using these measurements. The third involves measurements at only a particular subset of vertices, which nevertheless indirectly provide some information useful for inferring the unknown topology of the rest of the network graph. A visual characterization of these three types of problems is shown in Fig. 7.1.

7.2 Link Prediction

Let $\mathbf{Y}$ be the random $N_v \times N_v$ binary adjacency matrix for a network graph $G = (V,E)$. Suppose that we observe only some of the entries of $\mathbf{Y}$, while others are missing. Denote the observed and missing elements of $\mathbf{Y}$ as $\mathbf{Y}^{obs}$ and $\mathbf{Y}^{miss}$, respectively. The problem of *link prediction* is to predict the entries in $\mathbf{Y}^{miss}$, given the values $\mathbf{Y}^{obs} = \mathbf{y}^{obs}$ and possibly various vertex attribute variables $\mathbf{X} = \mathbf{x} = (x_1, \ldots, x_{N_v})^T$. In other words, we wish to predict whether 'potential edges' between pairs of vertices in a network graph are present or absent using information provided by a subset of observed edges/non-edges and, if available, vertex attributes.

There are a number of variants of the link prediction problem that have been formulated, arising in settings ranging from information networks (e.g., Liben-Nowell and Kleinberg [101], Popescul and Ungar [123], Taskar et al. [137]), to social networks (e.g., Hoff [72]), to biomolecular networks (e.g., Goldberg and Roth [65], Bader et al. [7]). Besides differing in context, these variants of the problem can also differ, importantly, in why and how the values in $\mathbf{Y}^{miss}$ are missing. Sometimes there is simply a temporal component to the problem and, for example, edges may be 'missing' only in the sense that they are absent up to a certain point in time and then become present, as in Liben-Nowell and Kleinberg [101] and their study of the growth of the World Wide Web network graph. In many cases, however, all edges and non-edges effectively coincide in time, but the status of potential edges is missing due to issues of sampling. In this latter case the underlying mechanism of missingness can be important.

A common assumption (e.g., Hoff [72], Taskar et al. [137]), and one we shall make here as well, is that the missing information on edge presence/absence is *missing at random*. This assumption means, essentially, that the probability that an edge

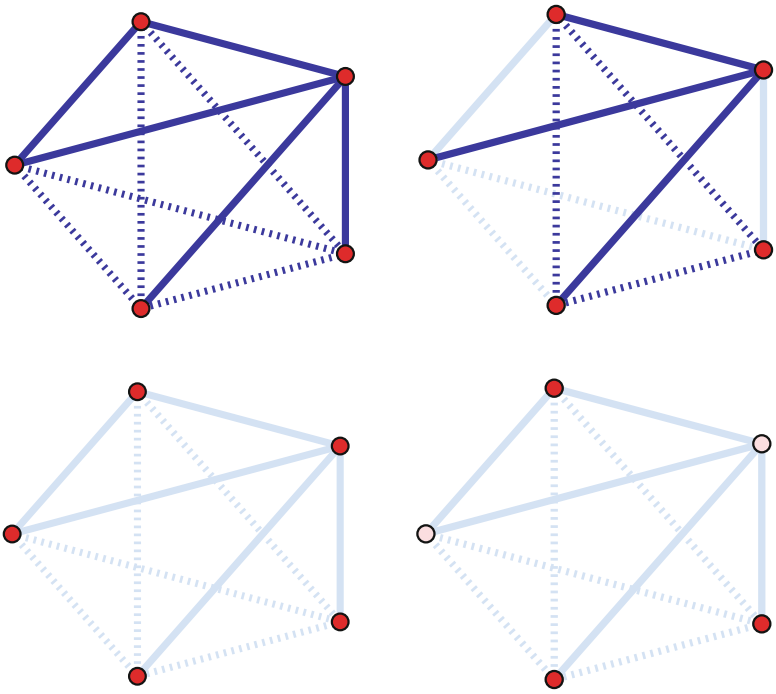

Fig. 7.1 Visual characterization of three types of network topology inference problems, for a toy network graph G. Edges shown in *solid*; non-edges, *dotted*. Observed vertices and edges shown in *dark* (i.e., *red* and *blue*, respectively); un-observed vertices and edges, in *light* (i.e., *pink* and *light blue*). *Top left*: True underlying graph G. *Top right*: Link prediction. *Bottom left*: Association network inference. *Bottom right*: Tomographic network inference

variable Y_{ij} is observed depends only on the values of those other edge variables observed and not, for example, on its own value.[1]

Given an appropriate model for $\mathbf{X}$ and $(\mathbf{Y}^{obs}, \mathbf{Y}^{miss})$, such as those discussed in Chap. 6, we might aim to jointly predict the elements of $\mathbf{Y}^{miss}$ based on the induced model for

$$\mathbb{P}\left(\mathbf{Y}^{miss} \,|\, \mathbf{Y}^{obs} = \mathbf{y}^{obs}, \mathbf{X} = \mathbf{x}\right). \tag{7.1}$$

But serious pursuit of this strategy entails, in part, successfully meeting the various challenges of modeling network graphs already described in Chap. 6, and in addition modeling the missingness in an appropriate fashion.

Perhaps not surprisingly, therefore, to date most model-based efforts in this area have instead focused upon the comparatively more manageable task of predicting the variables Y_{ij}^{miss} individually. Not only does this approach simplify matters

[1] See Little and Rubin [103], for example, for a more formal definition and a general introduction to such missing data concepts.

considerably in many ways, but it also appears to in fact be a strong competitor to methods that predict the variables Y_{ij}^{miss} jointly. See Taskar et al. [137], for example.

In fact, we have already seen this approach demonstrated in the use of cross-validation as a framework for assessing model goodness-of-fit, in Sect. 6.4. There the values that we leave out for each of the K folds are indeed missing-at-random, by design. And the predictions for the status of each potential edge between vertex pairs i and j were based on the posterior expectation of Y_{ij} under our model.

Alternatively, scoring methods—methods based on the use of score functions—although somewhat less formal than model-based approaches, are nevertheless popular and can be quite effective. With scoring methods, for each pair of vertices i and j whose edge status is unknown, a score $s(i,j)$ is computed. A set of predicted edges may then be returned either by applying a threshold s^* to these scores, for some fixed s^*, or by ordering them and keeping those pairs with the top n^* values, for some fixed n^*.

There are many types of scores that have been proposed in the literature. They generally are designed to assess certain structural characteristics of a network graph $G^{obs} = (V^{obs}, E^{obs})$ associated with $\mathbf{Y}^{obs} = \mathbf{y}^{obs}$. A simple score, inspired by the 'small-world' principle, is negative the shortest-path distance between i and j,

$$s(i,j) = -\,\mathsf{dist}_{G^{obs}}(i,j) \quad . \tag{7.2}$$

The negative sign in (7.2) is present so as to have larger score values indicate vertex pairs more likely to share an edge.

There are also a number of scores based on comparison of the observed neighborhoods $\mathcal{N}_i^{obs}$ and $\mathcal{N}_j^{obs}$ of i and j in G^{obs}, the simplest being the number of common neighbors

$$s(i,j) = |\mathcal{N}_i^{obs} \cap \mathcal{N}_j^{obs}| \quad . \tag{7.3}$$

To illustrate the potential of simple scoring methods like this, recall the network `fblog` of French political blogs. The number of nearest common neighbors for each pair of vertices in this network, excluding—if incident to each other—the two vertices themselves, may be computed in the following manner.

```
#7.1 1 > library(sand)
     2 > nv <- vcount(fblog)
     3 > ncn <- numeric()
     4 > A <- get.adjacency(fblog)
     5 >
     6 > for(i in (1:(nv-1))){
     7 +  ni <- neighborhood(fblog, 1, i)
     8 +  nj <- neighborhood(fblog, 1, (i+1):nv)
     9 +  nbhd.ij <- mapply(intersect, ni, nj, SIMPLIFY=FALSE)
    10 +  temp <- unlist(lapply(nbhd.ij, length)) -
    11 +    2 * A[i, (i+1):nv]
    12 +  ncn <- c(ncn, temp)
    13 + }
```

In Fig. 7.2 we compare the scores $s(i, j)$ for those vertex pairs that are not incident to each other (i.e., 'no edge'), and those that are (i.e., 'edge'), using so-called violin plots.[2]

```
#7.2 1 > library(vioplot)
     2 > Avec <- A[lower.tri(A)]
     3 > vioplot(ncn[Avec==0], ncn[Avec==1],
     4 +   names=c("No Edge", "Edge"))
     5 > title(ylab="Number of Common Neighbors")
```

It is evident from this comparison that there is a decided tendency towards larger scores when there is in fact an edge present. Viewing the calculation we have done here as a 'leave-one-out' cross-validation, and calculating the area under the curve (AUC) of the corresponding ROC curve,

```
#7.3 1 > library(ROCR)
     2 > pred <- prediction(ncn, Avec)
     3 > perf <- performance(pred, "auc")
     4 > slot(perf, "y.values")
     5 [[1]]
     6 [1] 0.9275179
```

we obtain further confirmation of the power of this score statistic to discriminate between edges and non-edges in this network.

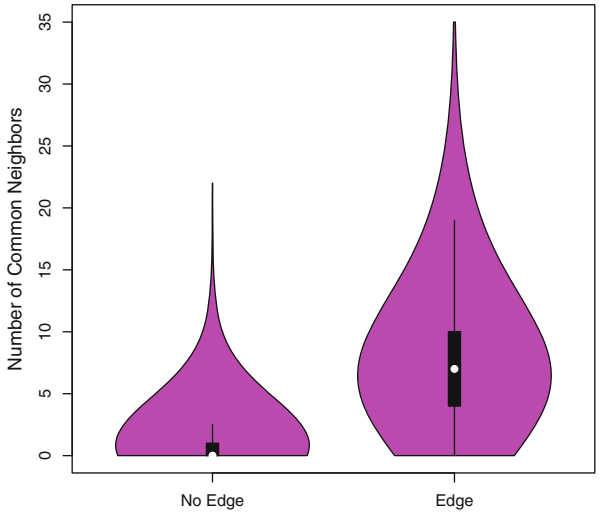

Fig. 7.2 Comparison of the number of common neighbors score statistic [i.e., (7.3)] in the network of French political blogs, grouped according to whether or not an edge is actually present between a vertex pair, for all vertex pairs

[2] These plots combine a traditional boxplot (in the middle) with a kernel density estimate, running in a symmetric fashion to either side.

Other choices of similar score statistics include a standardized version of this value, called the *Jaccard coefficient*,

$$s(i,j) = \frac{|\mathcal{N}_i^{obs} \cap \mathcal{N}_j^{obs}|}{|\mathcal{N}_i^{obs} \cup \mathcal{N}_j^{obs}|} \ , \tag{7.4}$$

and a variation on this idea due to Liben-Nowell and Kleinberg [101], extending a more general idea of Adamic and Adar [1], of the form

$$s(i,j) = \sum_{k \in \mathcal{N}_i^{obs} \cap \mathcal{N}_j^{obs}} \frac{1}{\log |\mathcal{N}_k^{obs}|} \ . \tag{7.5}$$

This last score in (7.5) has the effect of weighting more heavily those common neighbors of i and j that are themselves not highly connected.

7.3 Association Network Inference

Often the rule used for defining edges in the network graph representation of a given data set is that there be a sufficient level of 'association' between certain attributes of the two incident vertices. Such *association networks* are found in many domains and include networks of citation patterns across scientific articles, networks of actors co-starring in movies, networks of regulatory influence among genes, and networks of functional connectivity between regions of the brain.

Frequently the rules defining edges in such networks are specified without statistics necessarily playing an explicit role. While such rule-based approaches are appropriate in some contexts, in other contexts, where issues of sampling or measurement error are of potential concern, it may be necessary to incorporate statistical principles and methods into the process of constructing an association network graph. That is, the problem becomes one of association network inference.

Formally, we may suppose we have a collection of elements represented as vertices $v \in V$. Furthermore, suppose that each such vertex v has corresponding to it a vector $\mathbf{x}$ of m observed vertex attributes, yielding a collection $\{\mathbf{x}_1, \ldots, \mathbf{x}_{N_v}\}$ of attribute vectors. Let $\mathsf{sim}(i,j)$ be a user-specified quantification of the inherent similarity between the pair of vertices $i, j \in V$, and assume that it is accompanied by a corresponding notion of what value(s) of $\mathsf{sim}(i,j)$ constitute a 'non-trivial' level of association between i and j. We are interested here in the case where sim is not itself directly observable, but nevertheless the attributes $\{\mathbf{x}_i\}$ contain sufficiently useful information to make inference on sim conceivable.

There are of course countless choices for sim in practice. Here we concentrate on two common and popular linear measures of association—correlation and partial correlation—and the corresponding methods for inferring an association network based upon them.

We will illustrate these methods in the context of gene regulatory networks. The data in `Ecoli.expr` contain two objects.

```
#7.4 1 > rm(list=ls())
     2 > data(Ecoli.data)
     3 > ls()
     4 [1] "Ecoli.expr" "regDB.adj"
```

The first is a 40 by 153 matrix of (log) gene expression levels in the bacteria *Escherichia coli (E. coli)*, measured for 153 genes under each of 40 different experimental conditions, averaged over three replicates of each experiment. The data are a subset of those published in [54]. The genes are actually so-called transcription factors, which in general are known to play a particularly important role in the network of gene regulatory relationships. The experiments were genetic perturbation experiments, in which a given gene was 'turned off', for each of 40 different genes. The gene expression measurements are an indication of the level of activity of each of the 153 genes under each of the given experimental perturbations.

In Fig. 7.3 is shown a heatmap visualization of these data.

```
#7.5 1 > heatmap(scale(Ecoli.expr), Rowv=NA)
```

The genes (columns) have been ordered according to a hierarchical clustering of their corresponding vectors of expression levels. Due to the nature of the process of gene regulation, the expression levels of gene regulatory pairs often can be expected to vary together. We see some visual evidence of such associations in the figure, where certain genes show similar behavior across certain subsets of experiments. This fact suggests the value of constructing an association network from these data, as a proxy for the underlying network(s) of gene regulatory relationships at work, a common practice in systems biology.

The second object in this data set is an adjacency matrix summarizing our (incomplete) knowledge of actual regulatory relationships in *E. coli*, extracted from the RegulonDB database[3] at the same time the experimental data were collected. We coerce this matrix into a network object

```
#7.6 1 > library(igraph)
     2 > g.regDB <- graph.adjacency(regDB.adj, "undirected")
     3 > summary(g.regDB)
     4 IGRAPH UN-- 153 209 --
     5 attr: name (v/c)
```

and note that there are 209 known regulatory pairs represented. Visualizing this graph

```
#7.7 1 > plot(g.regDB, vertex.size=3, vertex.label=NA)
```

we see, examining the result in Fig. 7.4, that these pairs are largely contained within a single connected component.

The information in the graph `g.regDB` represents a type of aggregate truth set, in that some or all of these gene–gene regulatory relationships may be active in the biological organism under a given set of conditions. As such, the graph will be useful as a point of reference in evaluating on these data the methods of association network inference we introduce below.

[3] http://regulondb.ccg.unam.mx/.

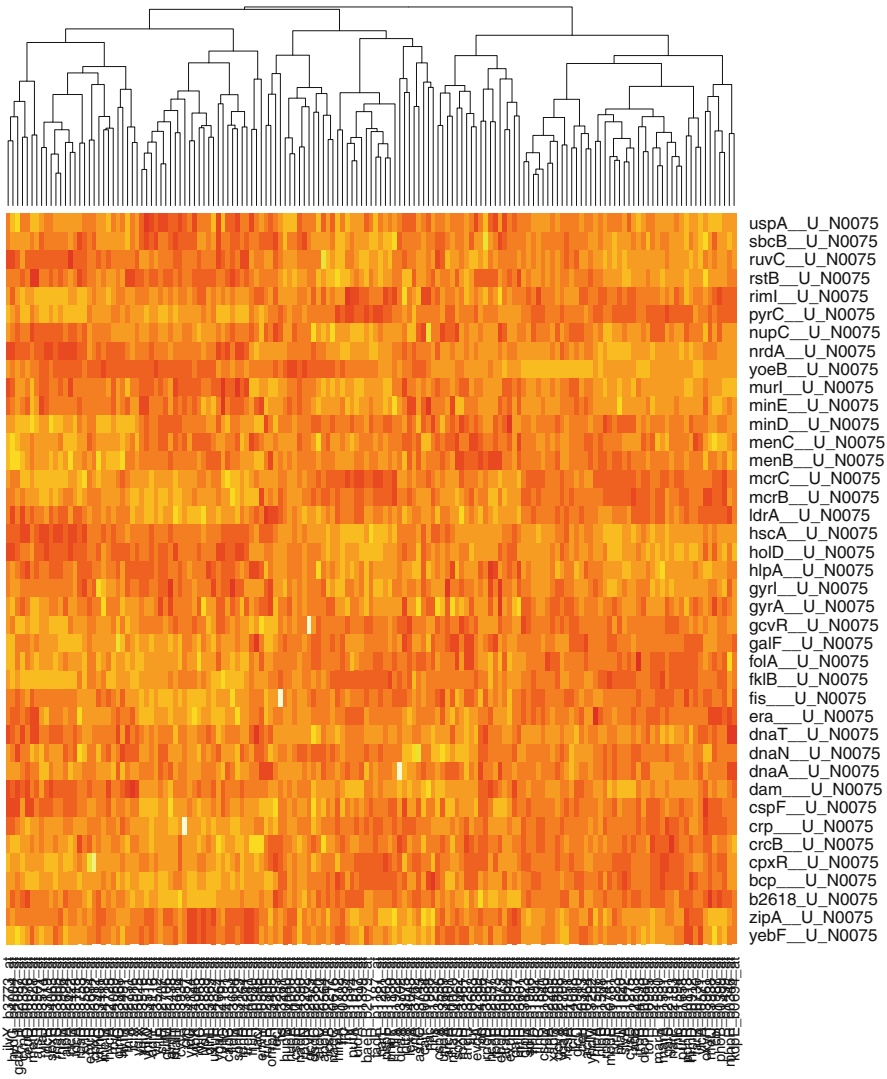

Fig. 7.3 Heatmap visualization of gene expression patterns over 40 experiments (*rows*) for 153 genes (*columns*)

7.3.1 Correlation Networks

Let X be a (continuous) random variable of interest corresponding to the vertices in V. A standard measure of similarity between vertex pairs is $\mathsf{sim}(i,j) = \rho_{ij}$, where

$$\rho_{ij} = \mathsf{corr}_{X_i,X_j} = \frac{\sigma_{ij}}{\sqrt{\sigma_{ii}\sigma_{jj}}} \tag{7.6}$$

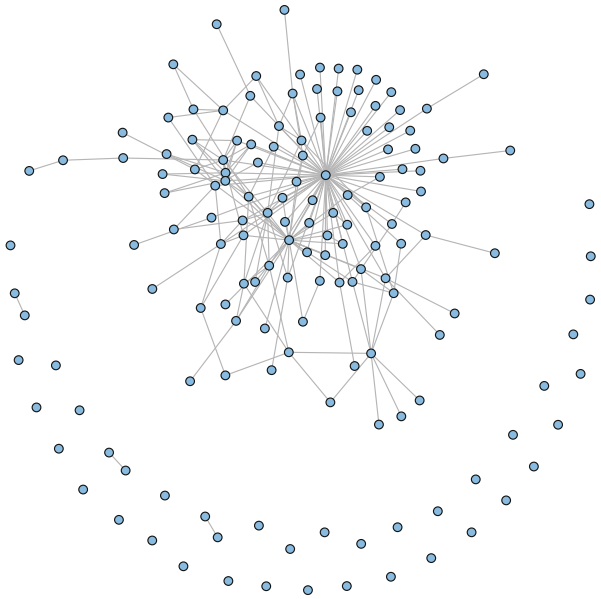

Fig. 7.4 Network of known regulatory relationships among 153 genes in *E. coli*

is the *Pearson product-moment correlation* between X_i and X_j, expressed in terms of the entries of the covariance matrix $\Sigma = \{\sigma_{ij}\}$ of the random vector $(X_1, \ldots, X_{N_v})^T$ of vertex attributes.

Given this choice of similarity, a natural criterion defining association between i and j is that ρ_{ij} be non-zero. The corresponding association graph G is then just the graph (V, E) with edge set

$$E = \left\{ \{i, j\} \in V^{(2)} : \rho_{ij} \neq 0 \right\} \quad . \tag{7.7}$$

This graph is often called a *covariance (correlation) graph*.

Given a set of observations of the X_i, the task of inferring this association network graph can be taken as equivalent to that of inferring the set of non-zero correlations. One way this task can be approached is through testing[4] the hypotheses

$$H_0 : \rho_{ij} = 0 \quad \text{versus} \quad H_1 : \rho_{ij} \neq 0 \quad . \tag{7.8}$$

However, there are at least three important issues to be faced in doing so. First, there is the choice of test statistic to be used. Second, given a test statistic, an appropriate null distribution must be determined for the evaluation of statistical significance. And third, there is the fact that a large number of tests are to be conducted simultaneously (i.e., for all $N_v(N_v - 1)/2$ potential edges), which implies that the problem of multiple testing must be addressed.

[4] Another way is through the use of penalized regression methods, similar to those that we discuss in Sect. 7.3.3.

Suppose that for each vertex $i \in V$, we have n independent observations $x_{i1}, \ldots, x_{in}$ of X_i. As test statistics, the *empirical correlations*

$$\hat{\rho}_{ij} = \frac{\hat{\sigma}_{ij}}{\sqrt{\hat{\sigma}_{ii}\hat{\sigma}_{jj}}} \tag{7.9}$$

are a common choice, where the $\hat{\sigma}_{ij}$ are the corresponding empirical versions of the σ_{ij} in (7.6).

To calculate these values across all gene pairs in our expression data is straightforward.

```
#7.8 1 > mycorr <- cor(Ecoli.expr)
```

However, it is more common to work with transformations of these values. For example, Fisher's transformation,

$$z_{ij} = \tanh^{-1}(\hat{\rho}_{ij}) = \frac{1}{2}\log\left[\frac{(1+\hat{\rho}_{ij})}{(1-\hat{\rho}_{ij})}\right] \tag{7.10}$$

can be helpful as a variance stabilizing transformation. If the pair of variables (X_i, X_j) has a bivariate normal distribution, the density of $\hat{\rho}_{ij}$ under $H_0 : \rho_{ij} = 0$ is known to be well approximated by that of a Gaussian random variable with mean zero and variance $1/(n-3)$, for sufficiently large n.

Accordingly, we transform the correlations in `mycorr` following (7.10),

```
#7.9 1 > z <- 0.5 * log((1 + mycorr) / (1 - mycorr))
```

and calculate p-values by comparing to the appropriate normal distribution.

```
#7.10 1 > z.vec <- z[upper.tri(z)]
      2 > n <- dim(Ecoli.expr)[1]
      3 > corr.pvals <- 2 * pnorm(abs(z.vec), 0,
      4 +   sqrt(1 / (n-3)), lower.tail=FALSE)
```

In assessing these p-values, however, it is necessary to account for the fact that we are conducting

```
#7.11 1 > length(corr.pvals)
      2 [1] 11628
```

tests simultaneously. The R function `p.adjust` may be used to calculate p-values adjusted for multiple testing. Here we apply a Benjamini-Hochberg adjustment, wherein p-values are adjusted through control of the false discovery rate.[5]

[5] The *false discovery rate* (FDR) is defined to be

$$\mathsf{FDR} = \mathbb{E}\left(\frac{R_{false}}{R}\,\middle|\, R > 0\right)\mathbb{P}(R > 0) \quad , \tag{7.11}$$

where R is the number of rejections among m tests and R_{false} is the number of false rejections. Benjamini and Hochberg [11] provide a method for controlling the FDR at a user-specified level γ by rejecting the null hypothesis for all tests associated with p-values $p_{(j)} \leq (j/m)\gamma$, where $p_{(j)}$ is the jth smallest p-value among the m tests.

```
#7.12 1 > corr.pvals.adj <- p.adjust(corr.pvals, "BH")
```

Comparing these values to a standard 0.05 significance level, we find a total of

```
#7.13 1 > length(corr.pvals.adj[corr.pvals.adj < 0.05])
      2 [1] 5227
```

gene pairs implicated. This number is far too large to be realistic, and an order of magnitude larger than in the aggregate network shown in Fig. 7.4. Simple diagnostics (e.g., using the `qqnorm` function to produce normal QQ plots) show that, unfortunately, the assumption of normality for the Fisher transformed correlations `z` is problematic with these data. As a result, any attempt to assign edge status to the 5227 'significant' vertex pairs above is highly suspect.

Fortunately, being in a decidedly 'data rich' situation, with over 11 thousand unique elements in `z`, we have the luxury to use data-dependent methods to learn the null distribution. The R package **fdrtool** implements several such methods.

```
#7.14 1 > library(fdrtool)
```

The methods in this package share the characteristic that they are all based on mixture models for some statistic S, of the form

$$f(s) = \eta_0 f_0(s) + (1 - \eta_0) f_1(s) \ , \tag{7.12}$$

where $f_0(s)$ denotes the probability density function of S under the null hypothesis H_0, and $f_1(s)$, under the alternative hypothesis H_1, with η_0 effectively serving as the fraction of true null hypotheses expected in the data. Both f_0 and f_1 are estimated from the data, as is the mixing parameter η_0. In this framework, multiple testing is controlled through control of various notions of false discovery rates.

Here, taking the statistic S to be the empirical correlation (7.9) and using the default options in the main function `fdrtool` within **fdrtool**,

```
#7.15 1 > mycorr.vec <- mycorr[upper.tri(mycorr)]
      2 > fdr <- fdrtool(mycorr.vec, statistic="correlation")
```

results in the output summarized in Fig. 7.5. Note that the histogram of (absolute) empirical correlation coefficients shows a single and widely dispersed mode whose shape, although little resembling a normal distribution, can be easily estimated from the data. Note too, however, that with an estimate of $\hat{\eta}_0 = 1$, the method is indicating that all of the correlation coefficients are judged to come from the single density function f_0. Accordingly, the false discovery rate is estimated to be 100 % no matter what threshold might be applied to the correlations.

Hence, using an empirically derived null distribution, which more accurately captures the behavior of the data in this case, we are led to the conclusion that the correlation network we seek to infer is in fact the empty graph. A more apt interpretation, however, is simply that the combination of (a) choice of similarity, (b) sample size, and (c) distribution of effect size together have made it impossible in this data to discern a clear difference between edges and non-edges. We shall see momentarily that the use of a more nuanced notion of similarity leads to decidedly different results.

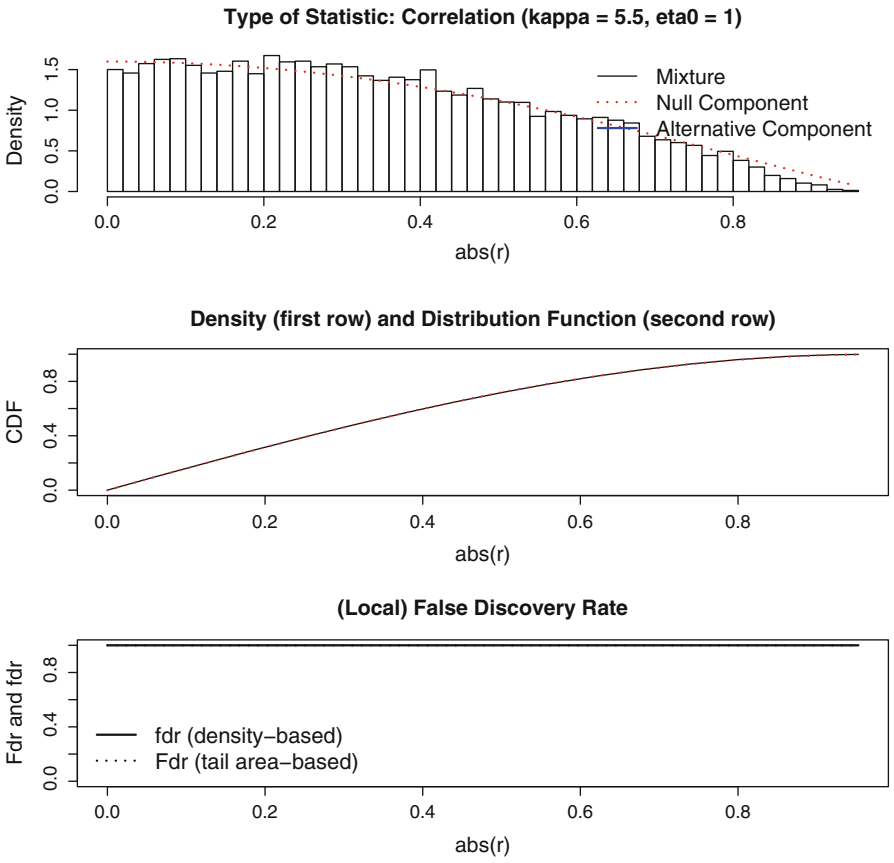

Fig. 7.5 Analysis of the empirical correlation coefficients for the gene expression data. *Top*: Estimated components f_0 and f_1 of the mixture in (7.12). *Middle*: Estimated density f. *Bottom*: False discovery rate

7.3.2 Partial Correlation Networks

The oft-cited dictum *'correlation does not imply causation'* should be kept in mind when constructing association networks based on Pearson's correlation and the like. Two vertices $i, j \in V$ may have highly correlated attributes X_i and X_j because the vertices somehow strongly 'influence' each other in a direct fashion. Alternatively, however, their correlation may be high primarily because, for example, they each are strongly influenced by a third vertex, say $k \in V$, and hence X_i and X_j are each highly correlated with X_k. The extent to which this issue is problematic or not will of course depend in no small part on the intended usage of the network graph G we seek to infer. But if it is felt desirable to construct a graph G where the inferred edges are more reflective of direct influence among vertices, rather than indirect influence, the notion of *partial correlation* becomes relevant.

In words, the partial correlation of attributes X_i and X_j of vertices $i, j \in V$, defined with respect to the attributes $X_{k_1}, \ldots, X_{k_m}$ of vertices $k_1, \ldots, k_m \in V \setminus \{i, j\}$, is the correlation between X_i and X_j left over after adjusting for those effects of $X_{k_1}, \ldots, X_{k_m}$ common to both. Formally, letting $S_m = \{k_1, \ldots, k_m\}$, we define the partial correlation of X_i and X_j, adjusting for $\mathbf{X}_{S_m} = (X_{k_1}, \ldots, X_{k_m})^T$, as

$$\rho_{ij|S_m} = \frac{\sigma_{ij|S_m}}{\sqrt{\sigma_{ii|S_m}\sigma_{jj|S_m}}} \quad . \tag{7.13}$$

Here $\sigma_{ii|S_m}, \sigma_{jj|S_m}$, and $\sigma_{ij|S_m} = \sigma_{ji|S_m}$ are the diagonal and off-diagonal elements, respectively, of the 2×2 partial covariance matrix

$$\Sigma_{11|2} = \Sigma_{11} - \Sigma_{12}\Sigma_{22}^{-1}\Sigma_{21} \quad , \tag{7.14}$$

where Σ_{11}, Σ_{22}, and $\Sigma_{12} = \Sigma_{21}^T$ are defined through the partitioned covariance matrix

$$\mathrm{Cov}\begin{pmatrix} \mathbf{W}_1 \\ \mathbf{W}_2 \end{pmatrix} = \begin{bmatrix} \Sigma_{11} & \Sigma_{12} \\ \Sigma_{21} & \Sigma_{22} \end{bmatrix} \quad , \tag{7.15}$$

for $\mathbf{W}_1 = (X_i, X_j)^T$ and $\mathbf{W}_2 = \mathbf{X}_{S_m}$. If $(X_i, X_j, X_{k_1}, \ldots, X_{k_m})^T$ has a multivariate Gaussian distribution, then $\rho_{ij|S_m} = 0$ if and only if X_i and X_j are independent conditional on $\mathbf{X}_{S_m}$. For more general distributions, however, zero partial correlation will not necessarily imply independence (the converse, of course, is still true).

Note that in the case of $m = 0$, the partial correlation in (7.13) reduces to the Pearson correlation in (7.6). In addition, there are recursive expressions that relate each mth order partial correlation coefficient to three $(m-1)$th order coefficients. For example, in the case of $m = 1$, the partial correlation of the attribute values X_i and X_j, for two vertices i and j, adjusted for the attribute value X_k of a third vertex k, is given by

$$\rho_{ij|k} = \frac{\rho_{ij} - \rho_{ik}\rho_{jk}}{\sqrt{\left(1 - \rho_{ik}^2\right)\left(1 - \rho_{jk}^2\right)}} \quad . \tag{7.16}$$

For the general case see, for example, Anderson [5, Chap. 2.5.3].

Partial correlations can be used in various ways for defining an association network graph G, with respect to vertex attributes $X_1, \ldots, X_{N_v}$. For example, for a given choice of m, we may dictate that an edge be present only when there is correlation between X_i and X_j regardless of which m other vertices are conditioned upon. That is,

$$E = \left\{ \{i, j\} \in V^{(2)} : \rho_{ij|S_m} \neq 0, \text{ for all } S_m \in V_{\setminus\{i,j\}}^{(m)} \right\} \quad , \tag{7.17}$$

where $V_{\setminus\{i,j\}}^{(m)}$ is the collection of all unordered subsets of m (distinct) vertices from $V \setminus \{i, j\}$. Other choices are clearly possible as well.

Under the definition of edges in (7.17), the problem of determining the presence or absence of a potential edge $\{i, j\}$ in G can be represented as one of testing

$$H_0 : \rho_{ij|S_m} = 0 \quad \text{for some} \quad S_m \in V_{\setminus\{i,j\}}^{(m)}$$

versus

$$H_1 : \rho_{ij|S_m} \neq 0 \quad \text{for all} \quad S_m \in V_{\setminus\{i,j\}}^{(m)} \ . \tag{7.18}$$

In order to infer an association network graph in this context, given measurements $x_{i1}, \ldots, x_{in}$ for each vertex $i \in V$, we must again select a test statistic, construct an appropriate null distribution, and adjust for multiple testing, as above.

Analogous to the empirical correlation coefficient $\hat{\rho}_{ij}$ in (7.9), the empirical partial correlation coefficient, $\hat{\rho}_{ij|S_m}$, is a natural estimate of $\rho_{ij|S_m}$. And, similarly, Fisher's transformation

$$z_{ij|S_m} = \frac{1}{2} \log \left[\frac{(1 + \hat{\rho}_{ij|S_m})}{(1 - \hat{\rho}_{ij|S_m})} \right] \tag{7.19}$$

is often used in place of the partial correlations themselves. Under the assumption that the joint distribution of attributes $(X_i, X_j, X_{k_1}, \ldots, X_{k_m})^T$ is a multivariate normal distribution, this statistic has an approximate normal distribution, with mean 0 and variance $1/(n-m-3)$.

In approaching the testing problem in (7.18), it can be convenient to consider it as a collection of smaller testing sub-problems of the form

$$H_0' : \rho_{ij|S_m} = 0 \quad \text{versus} \quad H_1' : \rho_{ij|S_m} \neq 0 \ , \tag{7.20}$$

for which $\hat{\rho}_{ij|S_m}$ is a natural test statistic. A test of (7.18) may then be constructed from the tests of the sub-problems (7.20) through aggregation. For example, Wille et al. [148] suggest combining the information in p-values $p_{ij|S_m}$, over all S_m, by defining

$$p_{ij,max} = \max \left\{ p_{ij|S_m} \ : \ S_m \in V_{\setminus\{i,j\}}^{(m)} \right\} \tag{7.21}$$

to be the p-value for the test of (7.18), where the $p_{ij|S_m}$ are the p-values for the tests of (7.20).

By way of illustration, recall the gene expression data `Ecoli.expr` that we analyzed in the previous section. We let $m = 1$, meaning that we define our network graph G that we will infer to be an association network where edges indicate correlation between the expression of gene pairs even after adjusting for the contributions of any other single gene. The following code utilizes the recursion in (7.16) to compute the corresponding empirical partial correlation coefficients, and approximates the distribution of the Fisher transformation of each such coefficient by a normal, with mean 0 and variance $1/(n-4)$. Finally, p-values are assigned according to (7.21).

#7.16
```
> pcorr.pvals <- matrix(0, dim(mycorr)[1],
+     dim(mycorr)[2])
> for(i in seq(1, 153)){
+   for(j in seq(1, 153)){
+     rowi <- mycorr[i, -c(i, j)]
```

```
 6 +     rowj <- mycorr[j, -c(i, j)]
 7 +     tmp <- (mycorr[i, j] -
 8 +       rowi*rowj)/sqrt((1-rowi^2) * (1-rowj^2))
 9 +     tmp.zvals <- (0.5) * log((1+tmp) / (1-tmp))
10 +     tmp.s.zvals <- sqrt(n-4) * tmp.zvals
11 +     tmp.pvals <- 2 * pnorm(abs(tmp.s.zvals),
12 +       0, 1, lower.tail=FALSE)
13 +     pcorr.pvals[i, j] <- max(tmp.pvals)
14 +   }
15 + }
```

Adjusting for multiple testing, as in the previous section,

```
#7.17 1 > pcorr.pvals.vec <- pcorr.pvals[lower.tri(pcorr.pvals)]
      2 > pcorr.pvals.adj <- p.adjust(pcorr.pvals.vec, "BH")
```

and applying a nominal threshold of 0.05, we see that a total of

```
#7.18 1 > pcorr.edges <- (pcorr.pvals.adj < 0.05)
      2 > length(pcorr.pvals.adj[pcorr.edges])
      3 [1] 25
```

edges have been discovered under this analysis. Creating the corresponding network graph

```
#7.19 1 > pcorr.A <- matrix(0, 153, 153)
      2 > pcorr.A[lower.tri(pcorr.A)] <- as.numeric(pcorr.edges)
      3 > g.pcorr <- graph.adjacency(pcorr.A, "undirected")
```

and comparing to the aggregate network in Fig. 7.4,

```
#7.20 1 > str(graph.intersection(g.regDB, g.pcorr, byname=FALSE))
      2 IGRAPH UN-- 153 4 --
      3 + attr: name (v/c)
      4 + edges (vertex names):
      5 [1] yhiW_b3515_at--yhiX_b3516_at
      6 [2] rhaR_b3906_at--rhaS_b3905_at
      7 [3] marA_b1531_at--marR_b1530_at
      8 [4] gutM_b2706_at--srlR_b2707_at
```

we find that four of these 25 edges are among those established in the biological literature.

Alternatively, the analogous analysis of these same data using `fdrtool`, fitting a mixture model directly now to the *p*-values in `pcorr.pvals.vec`, yields the same number of edges.

```
#7.21 1 > fdr <- fdrtool(pcorr.pvals.vec, statistic="pvalue",
      2 +   plot=FALSE)
      3 > pcorr.edges.2 <- (fdr$qval < 0.05)
      4 > length(fdr$qval[pcorr.edges.2])
      5 [1] 25
```

It may be easily verified that these are in fact the same edges as above, thus suggesting a certain robustness of our results.

7.3.3 Gaussian Graphical Model Networks

A special—and, indeed, popular—case of the use of partial correlation coefficients is when $m = N_v - 2$ and the attributes are assumed to have a multivariate Gaussian joint distribution. Here the partial correlation between attributes of two vertices is defined conditional upon the attribute information at *all* other vertices. Denoting these coefficients as $\rho_{ij|V\setminus\{i,j\}}$, under the Gaussian assumption the vertices $i, j \in V$ have partial correlation $\rho_{ij|V\setminus\{i,j\}} = 0$ if and only if X_i and X_j are conditionally independent given all of the other attributes. The graph $G = (V, E)$ with edge set

$$E = \left\{ \{i, j\} \in V^{(2)} : \rho_{ij|V\setminus\{i,j\}} \neq 0 \right\} \tag{7.22}$$

is called a *conditional independence graph*.[6] The overall model, combining the multivariate Gaussian distribution with the graph G, is called a *Gaussian graphical model.*

A useful result in the context of Gaussian graphical models is that the partial correlation coefficients may be expressed in the form

$$\rho_{ij|V\setminus\{i,j\}} = \frac{-\omega_{ij}}{\sqrt{\omega_{ii}\omega_{jj}}} \ , \tag{7.23}$$

where ω_{ij} is the (i, j)th entry of $\Omega = \Sigma^{-1}$, the inverse of the covariance matrix Σ of the vector $(X_1, \ldots, X_{N_v})^T$ of vertex attributes. See Lauritzen [97, Chap. 5.1] or Whittaker [147, Chap. 5.8], for example. The matrix Ω is known as the *concentration* or *precision* matrix, and its non-zero off-diagonal entries, occurring as they do in the numerator of (7.23), are linked in one-to-one correspondence with the edges in G. As a result, G also is sometimes referred to as a *concentration graph*.

The problem of inferring G from data in this context was originally termed the 'covariance selection problem' by Dempster [45]. Traditionally, recursive, likelihood-based procedures have been used that effectively test the hypotheses

$$H_0 : \rho_{ij|V\setminus\{i,j\}} = 0 \quad \text{versus} \quad H_1 : \rho_{ij|V\setminus\{i,j\}} \neq 0 \ , \tag{7.24}$$

using the empirical partial correlations $\hat{\rho}_{ij|V\setminus\{i,j\}}$, defined as in (7.23), but with the entries of $\hat{\Omega} = \hat{\Sigma}^{-1}$ in place of those of Ω, where $\hat{\Sigma}$ is the usual unbiased estimate of covariance. However, for large-scale network graphs, it has arguably become the standard to use penalized regression methods to infer G, exploiting a variation of the well-known connection between linear correlation and linear regression.

Suppose that the random vector $(X_1, \ldots, X_{N_v})^T$ of vertex attributes has a multivariate Gaussian distribution, with covariance Σ, as assumed above, and also zero mean. Then a standard result yields that, for the attribute X_i of a fixed vertex i, given the values of the attributes at the remaining vertices, say $\mathbf{X}^{(-i)} = (X_1, \ldots, X_{i-1}, X_{i+1}, \ldots, X_{N_v})^T$, its conditional expectation has the form

[6] Note that it is actually the non-edges in G that correspond to conditional independence, and the edges, to conditional dependence.

$$\mathbb{E}[X_i \,|\, \mathbf{X}^{(-i)} = \mathbf{x}^{(-i)}] = \left(\beta^{(-i)}\right)^T \mathbf{x}^{(-i)} \quad , \tag{7.25}$$

where $\beta^{(-i)}$ is an $(N_v - 1)$-length parameter vector. See, for example, Johnson and Wichern [84, Chap. 4]. Furthermore, importantly, the entries of $\beta^{(-i)}$ can be expressed in terms of the entries of the precision matrix $\boldsymbol{\Omega}$, as $\beta_j^{(-i)} = -\omega_{ij}/\omega_{ii}$. See Lauritzen [97, Appendix C]. Hence, a potential edge $\{i, j\}$ is in the edge set E defined by (7.22) if and only if $\beta_j^{(-i)}$ (and, therefore, also $\beta_i^{(-j)}$) is not equal to zero.

These observations suggest that inference of G be pursued via inference of the non-zero elements of $\beta^{(-i)}$ in (7.25), using regression-based methods of estimation and variable selection. In fact, it can be shown that the vector $\beta^{(-i)}$ is the solution to the optimization problem

$$\arg \min_{\tilde{\beta}:\tilde{\beta}_i=0} \mathbb{E}\left[\left(X_i - \left(\tilde{\beta}^{(-i)}\right)^T \mathbf{X}^{(-i)}\right)^2\right] \quad . \tag{7.26}$$

It is natural, therefore, to replace this problem by a corresponding least-squares optimization. However, because there will be $N_v - 1$ variables in the regression for each X_i, and in addition it may be the case that $n \ll N_v$, a penalized regression strategy is prudent.

Meinshausen and Bühlmann [110], for example, have suggested using estimates of the form

$$\hat{\beta}^{(-i)} = \arg \min_{\beta:\beta_i=0} \sum_{k=1}^{n} \left(x_{ik} - \left(\beta^{(-i)}\right)^T \mathbf{x}_k^{(-i)}\right)^2 + \lambda \sum_{j \neq i} \left|\beta_j^{(-i)}\right| \quad . \tag{7.27}$$

This strategy is based on the use of the Lasso method of Tibshirani [138], which not only estimates the coefficients in $\beta^{(-i)}$ but also forces values $\hat{\beta}_j^{(-i)} = 0$ where the association between X_i and X_j is felt to be too weak, with respect to the choice of penalty parameter λ. In other words, the Lasso methodology performs simultaneous estimation and variable selection, a characteristic due to the particular form of the penalty.

There are several practical concerns that must be addressed in order to put the above ideas into practice. These include (i) assessing the extent to which the data conform to a multivariate normal assumption, and (ii) choice of the penalty parameter λ. The R package **huge**—for 'high-dimensional, undirected graph estimation'—integrates solutions addressing both of these issues. The main function `huge` generates an initial set of estimates, following several pre-processing steps that seek to transform the data to have marginal distributions close to normal and to stabilize the overall estimation problem. The default method of estimation adopts the criterion in (7.27) above.

```
#7.22 1 > library(huge)
      2 > set.seed(42)
      3 > huge.out <- huge(Ecoli.expr)
```

The choice of penalty parameter can be determined by either of two procedures, the first being a so-called 'rotational information criterion' (i.e., in the spirit of more traditional criteria like AIC or BIC), and the second, a method based on principles of subsampling, called stability selection. The former procedure is understood to be prone to under-selection, while the latter, to over-selection.

Applying the former to our gene expression data returns an empty graph.

```
#7.23 1 > huge.opt <- huge.select(huge.out, criterion="ric")
      2 > summary(huge.opt$refit)
      3 153 x 153 sparse Matrix of class "dsCMatrix", with 0 entries
      4 [1] i j x
      5 <0 rows> (or 0-length row.names)
```

Conversely, applying the latter, we arrive at a network graph that is substantially more dense.

```
#7.24 1 > huge.opt <- huge.select(huge.out, criterion="stars")
      2 > g.huge <- graph.adjacency(huge.opt$refit, "undirected")
      3 > summary(g.huge)
      4 IGRAPH U--- 153 759 --
```

Comparisons indicate that this new network fully contains the network we obtained previously using partial correlation

```
#7.25 1 > str(graph.intersection(g.pcorr, g.huge))
      2 IGRAPH U--- 153 25 --
      3 + edges:
      4  [1] 145--146 144--146 112--125 112--113 109--138
      5  [6] 108--135  97--111  96--119  92--107  87--104
      6 [11]  86-- 87  84--129  81--137  72--141  70-- 75
      7 [16]  60--127  46-- 77  42-- 43  38--153  37-- 98
      8 [21]  27--123  21-- 33  12--135   9-- 90   3-- 60
```

and, moreover, contains 22 edges among those established in the biological literature.

```
#7.26 1 > str(graph.intersection(g.regDB, g.huge, byname=FALSE))
      2 IGRAPH UN-- 153 22 --
      3 + attr: name (v/c)
      4 + edges (vertex names):
      5  [1] yhiW_b3515_at--yhiX_b3516_at
      6  [2] tdcA_b3118_at--tdcR_b3119_at
      7  [3] rpoE_b2573_at--rpoH_b3461_at
      8  [4] rpoD_b3067_at--tyrR_b1323_at
      9  [5] rhaR_b3906_at--rhaS_b3905_at
     10  [6] nac_b1988_at --putA_b1014_at
     11  [7] marA_b1531_at--marR_b1530_at
     12  [8] malT_b3418_at--rpoD_b3067_at
     13  [9] hns_b1237_at --rcsB_b2217_at
     14 [10] hipA_b1507_at--hipB_b1508_at
     15 [11] himA_b1712_at--himD_b0912_at
     16 [12] gutM_b2706_at--srlR_b2707_at
     17 [13] fruR_b0080_at--mtlR_b3601_at
```

```
[14] flhD_b1892_at--lrhA_b2289_at
[15] crp_b3357_at --srlR_b2707_at
[16] crp_b3357_at --pdhR_b0113_at
[17] crp_b3357_at --oxyR_b3961_at
[18] crp_b3357_at --malT_b3418_at
[19] crp_b3357_at --galS_b2151_at
[20] caiF_b0034_at--rpoD_b3067_at
[21] caiF_b0034_at--hns_b1237_at
[22] arcA_b4401_at--lldR_b3604_at
```

7.4 Tomographic Network Topology Inference

Tomographic network topology inference is named in analogy to tomographic imaging[7] and refers to the inference of 'interior' components of a network—both vertices and edges—from data obtained at some subset of 'exterior' vertices. Here 'exterior' and 'interior' are somewhat relative terms and generally are used simply to distinguish between vertices where it is and is not possible (or, at least, not convenient) to obtain measurements. For example, in computer networks, desktop and laptop computers are typical instances of 'exterior' vertices, while Internet routers to which we do not have access are effectively 'interior' vertices.

With information only obtainable through measurements at 'exterior' vertices, the tomographic network topology inference problem can be expected to be quite difficult in general. This difficulty is primarily due to the fact that, for a given set of measurements, there are likely many network topologies that conceivably could have generated them, and without any further constraints on aspects like the number of internal vertices and edges, and the manner in which they connect to one another, we have no sensible way of choosing among these possible solutions.

In order to obtain useful solutions to such problems, therefore, additional information must be incorporated into the problem statement by way of model assumptions on the form of the internal structure to be inferred. A key structural simplification has been the restriction to inference of networks in the form of trees. In fact, nearly all work in this area to date has focused on this particular version of the problem.

7.4.1 Constraining the Problem: Tree Topologies

Recall that an (undirected) *tree* $T = (V_T, E_T)$ is a connected graph with no cycles. A *rooted tree* is a tree in which a vertex $r \in V_T$ has been singled out. The subset of vertices $R \subset V_T$ of degree one are called the *leaves*; we will refer to the vertices in $V \setminus \{\{r\} \cup R\}$ as the *internal vertices*. The edges in E_T are often referred to as *branches*.

[7] In the field of imaging, *tomography* refers to the process of imaging by sections, such as is done in medical imaging contexts like X-ray tomography or positron emission tomography (PET).

For example, in Fig. 7.6 is shown the logical topology, during a period of 2001, of that portion of the Internet leading from a desktop computer in the Electrical and Computer Engineering Department at Rice University to similar machines at ten other university locations. Specifically, two of the destination machines were located on a different sub-network at Rice, two at separate locations in Portugal, and six at four other universities in the United States. The original machine at Rice forms the root of this tree, in this representation, while the other ten machines form the leaves. These machines are 'exterior' in the sense that they would be known to, for example, their users. The other vertices, as well as the branches, are all 'internal' in the sense that these typically would not be known to a standard computer user. Hence, the learning of this internal structure (from appropriate measurements) is of natural interest.

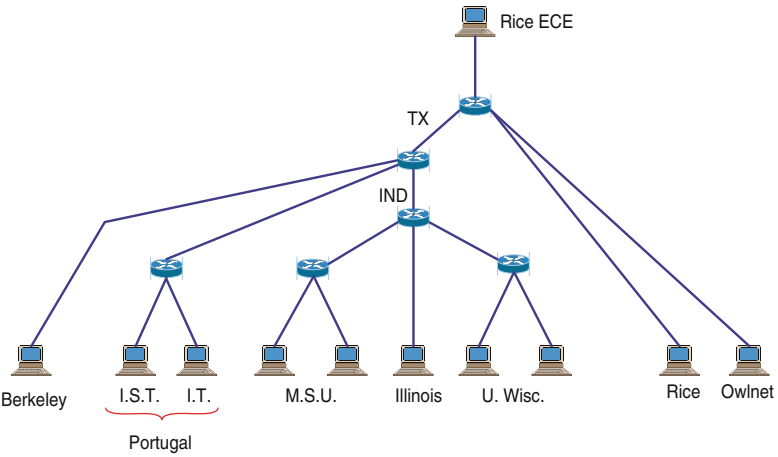

Fig. 7.6 Computer network topology from the experiment of Castro et al. [26], representing 'groundtruth'

It is quite common to restrict attention to binary trees. A *binary tree* is a rooted tree in which, in moving from the root towards the leaves, each internal vertex has at most two children. Trees with more general branching structure (such as that in Fig. 7.6) can always be represented as binary trees.

We define our tomographic network inference problem as follows. Suppose that for a set of N_l vertices—possibly a subset of the vertices V of an arbitrary graph $G = (V, E)$—we have n independent and identically distributed observations of some random variables $\{X_1, \ldots, X_{N_l}\}$. Under the assumption that these vertices can be meaningfully identified with the leaves R of a tree T, we aim to find that tree $\hat{T}$ in the set $\mathscr{T}_{N_l}$ of all binary trees with N_l labeled leaves that best explains the data, in some well-defined sense. If we have knowledge of a root r, then the roots of the trees in $\mathscr{T}_{N_l}$ will all be identified with r. In some contexts we may also be interested in inferring a set of weights for the branches in $\hat{T}$.

There are two main areas in which work on this type of network inference problem may be found. The first is in biology and, in particular, the inference of phylogenies. Phylogenetic inference is concerned with the construction of trees (i.e., phylogenies) from data, for the purpose of describing evolutionary relationships among biological species. See, for example, the book by Felsenstein [55]; also useful, and shorter, are the two surveys by Holmes [75, 76]. The second is in computer network analysis and the identification of logical topologies (i.e., as opposed to physical topologies). In computer network topology identification, the goal is to infer the tree formed by a set of paths along which traffic flows from a given origin Internet address to a set of destination addresses. Castro et al. [27, Sect. 4] provide a survey of this area.

The tree shown in Fig. 7.6 is an example of such a computer network topology. In order to learn such topologies (since, in general, a map of the Internet over such scales is not available), various techniques for probing the Internet have been developed. These probes, typically in the form of tiny packets of information, are sent from a source computer (e.g., the root machine at Rice University in Fig. 7.6) to various destinations (e.g., the leaf machines in Fig. 7.6), and information reflective of the underlying topology is measured.

For example, Coates et al. [33] propose a measurement scheme they call 'sandwich probing,' which sends a sequence of three packets, two small and one large. The large packet is sent second, after the first small packet and before the second small packet. The two small packets are sent to one of a pair $\{i, j\}$ of leaf vertices, say i, while the large packet is sent to the other, that is, j. The basic idea is that the large packet induces a greater delay in the arrival of the second small packet at its destination, compared to that of the first, and that the difference in delays of the two small packets will vary in a manner reflective of how much of the paths are shared from the origin to i and j.

The data set `sandwichprobe` contains the results of this experiment, consisting of (i) the measured delay between small packet pairs (in microseconds), indexed by the destination to which the small and large packets were sent, respectively, and (ii) the identifiers of destinations (called 'host' machines).

```
#7.27 1 > data(sandwichprobe)
      2 > delaydata[1:5, ]
      3   DelayDiff SmallPktDest BigPktDest
      4 1       757            3         10
      5 2       608            6          2
      6 3       242            8          9
      7 4        84            1          8
      8 5      1000            7          3
      9 > host.locs
     10  [1] "IST"    "IT"     "UCBkly" "MSU1"   "MSU2"
     11  [6] "UIUC"   "UW1"    "UW2"    "Rice1"  "Rice2"
```

Figure 7.7 shows an image representation of the mean delay differences, symmetrized to eliminate variations within each pair due to receipt of the small packets versus the large packet.

```
#7.28 1 > meanmat <- with(delaydata, by(DelayDiff,
      2 +   list(SmallPktDest, BigPktDest), mean))
      3 > image(log(meanmat + t(meanmat)), xaxt="n", yaxt="n",
      4 +   col=heat.colors(16))
      5 > mtext(side=1, text=host.locs,
      6 +   at=seq(0.0,1.0,0.11), las=3)
      7 > mtext(side=2, text=host.locs,
      8 +   at=seq(0.0,1.0,0.11), las=1)
```

The hierarchical relationship among the destinations in the logical topology in Fig. 7.6 is clearly evident in the relative magnitudes of the mean delay differences. This association can be used to infer the topology from the delay differences.

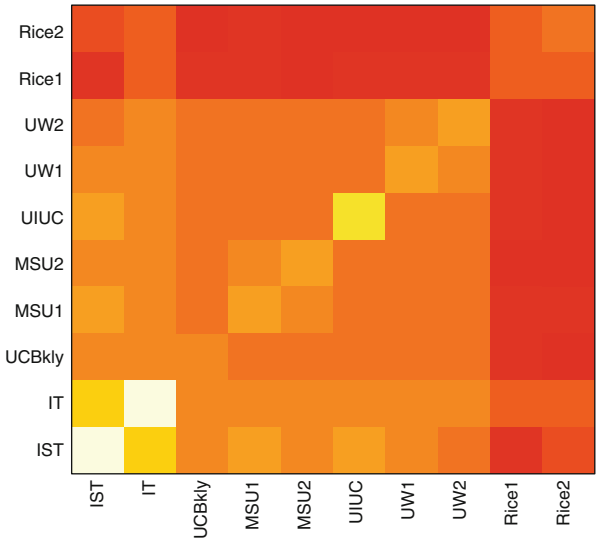

Fig. 7.7 Image representation of pairwise delay differences in the data of Coates et al. [33], with increasingly *darker red* corresponding to lower values, and increasingly *brighter yellow*, to higher values

7.4.2 Tomographic Inference of Tree Topologies: An Illustration

Broadly speaking, there are two classes of methods that have been developed quite extensively for the tomographic inference of tree topologies: (i) those based on hierarchical clustering and related ideas, and (ii) likelihood-based methods. Details associated with examples of the latter class of methods, being model-based, tend to be quite context-dependent. In contrast, simpler versions of the former class of methods are relatively straightforward to describe and may be implemented using

existing tools in the R base package. Therefore, although a treatment of likelihood-based methods is beyond the scope of this book,[8] we will take a look at a simple example of how hierarchical clustering may be applied to this problem.

Given a set of N_l objects, recall from our discussion in Chap. 4.4.1 that hierarchical clustering is concerned with the grouping of those objects into hierarchically nested sets of partitions, based on some notion of their (dis)similarity. And, in particular, recall that generally these nested partitions are represented using a tree. The usage of hierarchical clustering we saw in Chap. 4.4.1 was for the purpose of graph clustering. But it also is a natural tool to use for the tomographic inference of tree topologies, where we treat the N_l leaves as the 'objects' to be clustered and the tree corresponding to the resulting clustering as our inferred tree $\hat{T}$. It is perhaps worth emphasizing that, whereas in standard applications of hierarchical clustering the goal often ultimately is to select just one of the nested partitions to represent the data, here the tree corresponding to the entire set of partitions is our focus.

Recall that hierarchical clustering methods require some notion of (dis)similarity, which is usually constructed from the observed data, but sometimes measured directly. In the context of our tomographic inference problem, it is logical to use the n observations of the random variables $\{X_1, \ldots, X_{N_l}\}$ at the leaves to derive an $N_l \times N_l$ matrix of (dis)similarities.

For example, the information in the matrix `delaydata` may be used to compute the (squared) Euclidean distance between delays at each pair of destinations.

```
#7.29 1 > SSDelayDiff <- with(delaydata, by(DelayDiff^2,
      2 +   list(SmallPktDest, BigPktDest), sum))
```

The R function `hclust` may then be used for hierarchical clustering.

```
#7.30 1 > x <- as.dist(1 / sqrt(SSDelayDiff))
      2 > myclust <- hclust(x, method="average")
```

Here we have used the inverse of distance between delays as a similarity matrix. The choice of `average` for the hierarchical clustering method corresponds to using the unweighted pair group method with arithmetic mean (UPGMA), a standard choice and one that is quite similar to the more application-specific agglomerative likelihood tree (ALT) method of Coates et al. [33].

The result of our hierarchical clustering is displayed in Fig. 7.8, in the form of a dendrogram.

```
#7.31 1 > plot(myclust, labels=host.locs, axes=FALSE,
      2 +   ylab=NULL, ann=FALSE)
```

Comparing our inferred topology to the ground truth in Fig. 7.6, we see that many aspects of the latter are capture by the former. For example, the fact that the two destinations at Rice University are, naturally, much closer to the root machine at Rice than any of the other destinations is strongly indicated. Similarly, the two

[8] A brief treatment of such methods, largely from the perspective of computer network topology inference, may be found in [91, Chap. 7.4]. In terms of software, there are a number of fairly comprehensive packages in R dedicated to phylogenetic tree inference. See, for example, the packages **ape** and **phangorn**.

machines in Portugal are found grouped together, as are the two machines at the University of Wisconsin, and the three other machines in the midwest (i.e., two at Michigan State, and one at the University of Illinois). Curiously, however, the machine at Berkeley is inferred to be close to those at Wisconsin, although in reality they appear to be well-separated in the actual topology.

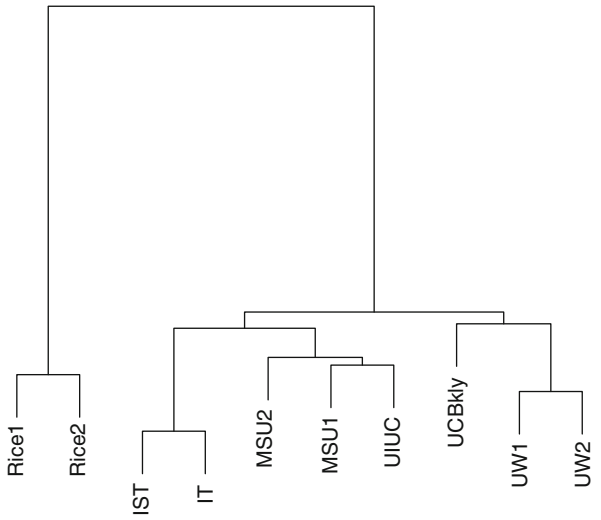

Fig. 7.8 Inferred topology, based on 'sandwich' probe measurements, using a hierarchical clustering algorithm

7.5 Additional Reading

Recent surveys of methods of link prediction include that of Lu and Zhou [104], in the physics literature, and Al Hasan and Zaki [3], in the computer science literature. For the specific problem of inferring gene regulatory networks, there are many resources to be found, including the recent survey of Noor et al. [120]. Finally, for more information on tomographic inference of tree topologies, the literature in phylogenetics is extensive. See, for example, the book by Felsenstein [55] or the surveys by Holmes [75, 76].